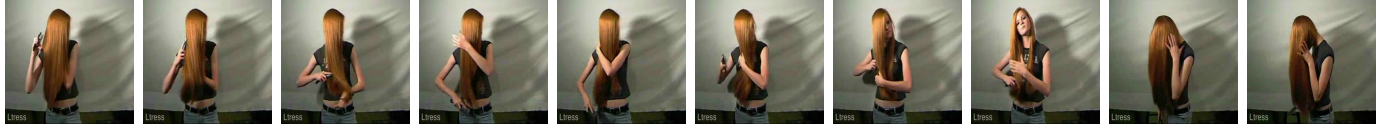


## EEEM068: Applied Machine Learning

### Project: Action Recognition using Vision Transformers

- Submission deadline: TBC

Brush Hair



Catch



Climb Stairs



Golf



Shoot Gun



Figure 1: Video frames from 5 of the 25 classes of the HMDB\_simp dataset.

## Project specification

### Overview

This project requires you to apply the knowledge gained from the tutorials of the EEEM068 Applied Machine Learning module. It accounts for 100% of the total module marks, distributed as follows: 60% for the working code and project report, and 40% for the oral exam and viva.

The project spans two complementary tasks that together reflect the full pipeline of video understanding. First, you will train and critically compare multiple Vision Transformer (ViT) architectures for **clip-level action classification** on the HMDB\_simp dataset. Second, you will extend your best-performing recognition model to the problem of **spatio-temporal action localisation**: given an untrimmed or multi-actor video, the system must not only identify *what* action is occurring but also *where* in the frame and *when* in the video it takes place. By tackling both tasks, you will gain a holistic understanding of how spatial, temporal, and spatiotemporal representations interact within transformer-based video models.

### Submission

You should submit a zip file containing the following:

- Your complete code;
- A **5-page report** (IEEE double-column format) explaining your code, visualizing the results you obtained, and discussing your observations. If you want, you can include additional pages only with visualizations (this is optional and you won't lose any marks if you do not include additional pages). However, the main text of your report should fit in the first 4 pages and the additional pages (if any) should only include visualizations with short captions.

Please note that for all the visualizations and tables you include in your report, it is important to include a reference in the main text (typically using a Figure or Table number).

## Background

In this project, you will use Vision Transformer (ViT) architectures first to *recognise* 25 actions in video data, and then to *localise* those actions in space and time. ViTs are state-of-the-art deep learning models with excellent performance on image and video understanding tasks. Unlike 3D CNNs that process raw spatiotemporal volumes directly, ViT-based video models decompose the problem into spatial and temporal reasoning through attention mechanisms — a property that also makes them naturally suited to localisation, since attention weights can indicate *where* and *when* the model is “looking”.

The project is structured as a progressive pipeline. You will begin with clip-level classification (recognising the action label for a short, pre-trimmed clip), then build on those learned representations to address spatio-temporal localisation (predicting bounding boxes around the actor and identifying the temporal extent of an action within longer sequences). This progression mirrors the research trajectory of the field and will give you experience with both recognition and detection paradigms.

You are expected to go beyond simply fine-tuning a single pre-trained model. This project requires you to **compare** at least two distinct transformer-based architectures, conduct **ablation studies** over key design choices, provide **inter-pretability analysis** of the learned spatiotemporal representations, and extend the best model to **spatio-temporal localisation**.

## Video datasets

**HMDB\_simp (clip-level classification).** You will use the same video dataset as in the original exercise. Each subfolder corresponds to a different action category. The dataset includes 1,250 videos, with 50 videos per category across 25 classes.

**JHMDB (spatio-temporal localisation).** For the localisation task you will use the **JHMDB** (Joint-annotated HMDB) dataset<sup>1</sup>, which provides per-frame bounding box annotations and joint-level pose data for 21 action categories across 928 video clips. JHMDB is a natural companion to HMDB\_simp because the two datasets share the same action taxonomy and visual domain, which means representations learned during classification can directly transfer to the localisation stage. The dataset and annotations can be obtained from <http://jhmdb.is.tue.mpg.de/>. You should use the standard train/test splits provided with the dataset.

## Part A: Multi-Architecture Comparison for Classification

You must implement, fine-tune, and compare **at least two** of the following transformer-based video models on the HMDB\_simp dataset:

1. **TimeSFormer** (Bertasius *et al.*, 2021)<sup>2</sup> — Uses divided space-time attention.
2. **VideoMAE** (Tong *et al.*, 2022)<sup>3</sup> — Uses masked autoencoder pre-training for video.
3. **ViViT** (Arnab *et al.*, 2021)<sup>4</sup> — Factorizes spatial and temporal dimensions of video.

Pre-trained checkpoints are available through the `transformers` library. For example, TimeSFormer can be loaded as follows:

---

```
from transformers import AutoFeatureExtractor, AutoModelForVideoClassification
```

```
extractor = AutoFeatureExtractor.from_pretrained("facebook/timesformer-base-finetuned-k400")  
model = AutoModelForVideoClassification.from_pretrained("facebook/timesformer-base-finetuned-k400")
```

---

Similarly, VideoMAE and ViViT have Hugging Face checkpoints (e.g., `MCG-NJU/videomae-base-finetuned-kinetics`, `google/vivit-b-16x2-kinetics400`). Refer to the Hugging Face documentation for implementation details.

For each model, you should:

- Adapt the classification head to output 25 classes.
- Use the model-specific pre-processing pipeline (frame sampling rate, spatial resolution, clip length).
- Report all metrics defined in Part C.

---

<sup>1</sup>Jhuang, H., Gall, J., Zuffi, S., Schmid, C. and Black, M.J., 2013. Towards understanding action recognition. In ICCV (pp. 3192–3199).

<sup>2</sup>Bertasius, G., Wang, H. and Torresani, L., 2021, July. Is Space-Time Attention All You Need for Video Understanding?. In ICML (pp. 813–824). PMLR.

<sup>3</sup>Tong, Z., Song, Y., Wang, J. and Wang, L., 2022. VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-training. In NeurIPS.

<sup>4</sup>Arnab, A., Dehghani, M., Heigold, G., Sun, C., Lučić, M. and Schmid, C., 2021. ViViT: A Video Vision Transformer. In ICCV (pp. 6836–6846).

- Provide a comparative analysis in terms of accuracy, computational cost (FLOPs and/or wall-clock training time per epoch), and number of trainable parameters.

When performing this comparison, pay attention to how each architecture’s attention mechanism handles the spatial and temporal dimensions differently. These architectural differences will become directly relevant in Part D, where the quality of spatiotemporal representations determines localisation performance.

## Part B: Ablation Studies

For your **best-performing** classification model from Part A, you must conduct ablation studies over at least **three** of the following factors. Each ablation should vary one factor while holding all others constant:

1. **Temporal sampling strategy:** Compare uniform sampling, random sampling, and dense sampling of frames from each video. How does the temporal coverage affect recognition of actions with different temporal extents? Consider also how each strategy would affect downstream localisation — does dense sampling help the model learn more precise temporal boundaries?
2. **Number of input frames:** Vary the number of frames per clip (e.g., 4, 8, 16) and analyze the trade-off between temporal context and computational cost. Relate your findings to the temporal granularity needed for accurate action boundary prediction in Part D.
3. **Fine-tuning strategy:** Compare (a) training only the classification head (linear probing), (b) fine-tuning the last  $k$  transformer blocks plus the head, and (c) full fine-tuning of all parameters. Discuss the effect on convergence speed, final accuracy, and overfitting risk given the limited dataset size. Note which strategy produces the most transferable features for the localisation task.
4. **Data augmentation:** Evaluate the impact of spatial augmentations (random cropping, horizontal flipping, color jitter) and temporal augmentations (temporal jitter, speed perturbation, reverse playback) on generalization. Spatial augmentations are particularly interesting in the context of localisation, since aggressive cropping may remove the actor of interest.
5. **Learning rate schedule:** Compare at least two scheduling strategies (e.g., cosine annealing, step decay, linear warmup + cosine decay) and report the effect on training dynamics and final performance.

Present ablation results in a clearly formatted table with accompanying discussion.

## Part C: Training, Evaluation, and Metrics

### C.1 Classification (HMDB\_simp)

- Similar to Worksheet 5, load the “HMDB\_simp” dataset by implementing an appropriate dataset and dataloader.
- Use the pre-processing steps specified for each model. For TimeSFormer, this includes taking clips of size  $8 \times 224 \times 224$ , with frames sampled at a rate of 1/32. Refer to the respective papers and Hugging Face documentation for each model’s requirements.
- Use **stratified** random splitting to divide the dataset into training (70%), validation (15%), and test (15%) sets. The stratified split ensures that each class is proportionally represented in all three splits. The validation set should be used for hyperparameter selection and early stopping; the test set should be used **only** for final reporting of results.
- Train each model with appropriate hyperparameters. Plot the training and validation loss curves and accuracy curves using `tensorboard` or `matplotlib`.
- For the final evaluation on the test set, report the following metrics:
  - Top-1 and Top-5 classification accuracy.
  - Per-class Precision, Recall, and F1-score (present in a table).
  - Macro-averaged and weighted-averaged F1-score.
  - Confusion matrix, visualized as a heatmap with class labels.
- Perform a **statistical robustness check**: run your best model configuration with at least 3 different random seeds, and report mean  $\pm$  standard deviation for top-1 and top-5 accuracy.

Please note that on the test set, the top-1 accuracy should be at least 45%, and the top-5 accuracy should be at least 75%.

## C.2 Spatio-Temporal Localisation (JHMDB)

- Load the JHMDB dataset with its per-frame bounding box annotations. Implement a dataloader that returns video clips together with their corresponding frame-level spatial annotations (bounding boxes) and action labels.
- Use the standard train/test splits provided with the JHMDB dataset.
- For evaluation, report the following localisation metrics:
  - **Frame-level mAP**: mean Average Precision computed on individual frames, where a detection is correct if the predicted bounding box has an  $\text{IoU} \geq 0.5$  with the ground-truth box *and* the predicted action label is correct.
  - **Video-level mAP**: mean Average Precision at the video tube level, where an action tube (a sequence of linked bounding boxes across frames) is correct if it has a spatio-temporal  $\text{IoU} \geq 0.5$  with the ground-truth tube and the predicted label is correct. You may use the evaluation protocol from Kalogeiton *et al.*<sup>5</sup>
  - Per-class breakdown of frame-mAP for at least the 5 best and 5 worst performing classes.
- Visualise qualitative localisation results: for at least 3 test videos, overlay the predicted bounding boxes (colour-coded by predicted class) alongside the ground-truth boxes on the video frames. Display these as a temporal strip (sampled frames from the clip arranged left to right) so that both spatial accuracy and temporal consistency of the detections are visible.

## Part D: Spatio-Temporal Localisation Pipeline

This part extends your best classification model from Part A to perform spatio-temporal action localisation on the JHMDB dataset. The goal is to produce *action tubes* — sequences of per-frame bounding boxes that track an action through the video — together with an action class label and a confidence score.

You should implement **one** of the following approaches (listed in order of increasing complexity):

### Option 1: Feature extraction + detection head (recommended starting point)

Use the frozen or lightly fine-tuned transformer backbone from Part A as a feature extractor. For each frame (or each spatiotemporal patch), extract the token-level features from an intermediate or final transformer layer. Attach a lightweight detection head on top of these features to predict:

- A bounding box regression ( $x, y, w, h$ ) for the actor in each frame.
- An action class probability distribution.

A simple approach is to reshape the spatial patch tokens back into a 2D feature map and apply a small convolutional detection head (e.g., 2–3 convolutional layers followed by regression and classification outputs), similar in spirit to a single-scale feature pyramid. To form action tubes, link per-frame detections temporally using a simple greedy strategy (e.g., highest IoU linking between consecutive frames).

*Hint*: The spatial tokens of a ViT naturally correspond to spatial locations in the input; a token at position  $(i, j)$  in the patch grid “sees” a specific image region, which gives you a coarse spatial prior even before adding a detection head.

### Option 2: Attention-guided localisation

Leverage the internal attention maps of the transformer to generate spatial localisation without an explicit detection head. Specifically:

- Extract the spatial attention maps (e.g., from the [CLS] token attending to spatial patch tokens) at one or more transformer layers.
- Threshold or refine the attention map to produce a binary mask per frame, then fit a bounding box to the largest connected component.
- Optionally apply attention rollout<sup>6</sup> across layers to obtain a more holistic localisation signal.
- Link the per-frame boxes into action tubes as in Option 1.

This option directly connects the interpretability analysis (Part E) with the localisation task: the same attention maps you visualise for interpretability become the mechanism by which the model localises actions.

<sup>5</sup>Kalogeiton, V., Weinzaepfel, P., Ferrari, V. and Schmid, C., 2017. Action Tubelet Detector for Spatio-Temporal Action Localization. In ICCV (pp. 4405–4413).

<sup>6</sup>Abnar, S. and Zuidema, W., 2020. Quantifying Attention Flow in Transformers. In ACL (pp. 4190–4197).

### Option 3: End-to-end spatio-temporal detector (advanced)

Implement or adapt a transformer-based spatio-temporal action detector that jointly performs feature extraction, classification, and bounding box regression in a single forward pass. Possible architectures include:

- A DETR-style<sup>7</sup> approach where a set of learnable action queries attend to the spatiotemporal feature tokens and directly output bounding boxes, class labels, and temporal extents via a set prediction loss (Hungarian matching + bipartite assignment).
- A TubeR-style<sup>8</sup> architecture that predicts action tubelets (short sequences of boxes) end-to-end.

This is the most challenging option and is particularly recommended for groups aiming for the highest marks or extra credit. If you choose this option, you may initialise the backbone from your Part A checkpoint and focus your contribution on the detection head and the training procedure.

**Regardless of the option chosen, you should:**

- Clearly describe the architecture of your localisation pipeline in the report, including a diagram.
- Explain how the transformer features from Part A are repurposed or extended for localisation.
- Report the metrics specified in Section C.2.
- Discuss where the localisation pipeline succeeds and fails, and how these failure cases relate to the classification errors observed in Part A (e.g., does a class that is frequently misclassified also produce poor localisations?).

## Part E: Interpretability, Visualisation, and Error Analysis

You must provide at least **two** of the following interpretability analyses. Note that several of these analyses serve a dual purpose: they help you understand the classification model *and* they directly inform or improve your localisation pipeline (Part D).

1. **Attention map visualisation:** Extract and visualise the spatial and/or temporal attention maps from one or more transformer layers. Overlay the attention weights on the input frames to show which spatial regions and temporal positions the model attends to for different action classes. Compare attention patterns between correctly and incorrectly classified examples. If you chose localisation Option 2, discuss whether the attention maps that your localisation pipeline uses are consistent with the attention maps visualised here.
2. **Feature space visualisation:** Use t-SNE or UMAP to project the learned video-level feature representations (e.g., the [CLS] token embedding from the final transformer layer) into 2D space. Colour-code the points by class label and discuss whether semantically similar actions cluster together. Additionally, project the features from *both* the HMDB-simp and JHMDB datasets into the same space and analyze whether the shared action classes overlap in feature space, which speaks to the transferability of representations between the classification and localisation tasks.
3. **Temporal contribution analysis:** For a subset of test videos, analyze how the model’s prediction confidence evolves as more frames are revealed (i.e., mask later frames and observe how the predicted class probability changes). This reveals which temporal segments are most discriminative for each action. Relate these findings to the temporal extent of the ground-truth action annotations in JHMDB: do the most discriminative temporal segments coincide with the annotated action boundaries?
4. **Grad-CAM or gradient-based saliency:** Apply Grad-CAM (or a suitable gradient-based method adapted for transformers, such as attention rollout or LRP) to visualise which spatial regions in key frames contribute most to the classification decision. Compare the resulting saliency maps with the predicted bounding boxes from Part D — do the salient regions align with the localised actor?

### Error analysis

Beyond reporting aggregate metrics, you must perform a structured error analysis that spans both the classification and localisation tasks:

- Identify the most commonly confused class pairs from the classification confusion matrix and hypothesise why these confusions occur (e.g., visual similarity, shared motion patterns, background bias).
- For your best classification model, randomly select at least 5 correctly classified and 5 misclassified test videos from HMDB-simp. For each, display sampled frames alongside the model’s top-5 predicted classes and their confidence scores. Discuss patterns in the errors.

---

<sup>7</sup>Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A. and Zagoruyko, S., 2020. End-to-End Object Detection with Transformers. In ECCV (pp. 213–229).

<sup>8</sup>Zhao, Y., Krähenbühl, P. and Torresani, L., 2022. TubeR: Tubelet Transformer for Video Action Detection. In CVPR (pp. 13598–13607).

- For the localisation task on JHMDB, identify at least 3 examples of accurate localisation and 3 examples of localisation failure. For each failure, diagnose whether the error is primarily *spatial* (the bounding box misses the actor), *temporal* (the action boundaries are incorrect), *categorical* (the box is on the right actor but the class label is wrong), or a combination. Visualise these cases with frame strips showing predicted vs. ground-truth boxes.
- Analyze whether there is a correlation between video-level properties (e.g., clip length, amount of motion, number of visible actors, scene complexity) and classification/localisation performance. A simple analysis (e.g., scatter plot of motion magnitude vs. frame-mAP, or bar chart of per-class mAP vs. average actor size) is sufficient.

## Discuss your Observations

- In your report, you should provide a coherent narrative that ties together your findings across all parts. Discuss observations with reference to the architectural comparison, ablation results, fine-tuning of hyperparameters, training behaviour, classification accuracy measures, localisation mAP, confusion matrices, interpretability visualisations, and error analysis. Whenever appropriate, please refer to the corresponding figures/tables to make your observations more concrete.
- Critically assess the strengths and weaknesses of the compared architectures for this specific dataset and task. Are there action categories that consistently benefit from one architecture over another? Does the architecture that achieves the best classification accuracy also produce the best features for localisation, or is there a disconnect?
- Reflect on the relationship between classification and localisation performance. For instance: do the classes with the highest classification F1-scores also achieve the highest frame-mAP in JHMDB? What does this tell you about the nature of the learned representations?

## Extra credit

Extra credit will be awarded for additional contributions that go meaningfully beyond the required components. Examples include, but are not limited to:

- Implementing an **ensemble** of your compared models (e.g., late fusion of logits or feature-level fusion) and demonstrating improved accuracy over individual models on both classification and localisation.
- Applying **knowledge distillation** from a larger model to a smaller one and analysing the accuracy–efficiency trade-off for both tasks.
- Building a simple **interactive demo** (e.g., Gradio or Streamlit interface) that takes a video as input and displays the predicted action label, confidence scores, and overlaid bounding boxes in real time.
- Performing a **cross-dataset transfer** evaluation (e.g., training the localisation head on JHMDB and evaluating zero-shot on UCF101-24) to assess generalisation of the spatio-temporal representations.
- Implementing a **multi-scale temporal localisation** approach: given a longer, untrimmed video composed of several concatenated HMDB\_simp clips, predict both the temporal boundaries and action labels of each segment (a temporal action detection problem).
- Conducting a **failure mode taxonomy** that categorises both classification and localisation errors into systematic types (e.g., background bias, temporal aliasing, viewpoint sensitivity, occlusion, small actor size) with quantitative evidence.

## Marking Criteria

The assessment for this project will be divided into two main components:

- **Group Coursework Submission (60%)**: Includes the technical report and working code.
- **Group Presentation and Oral Exam/Viva (40%)**: The presentation will be 5 minutes long, followed by a 15-minute Q&A session with questions directed at each group member.

## Group Requirements

- Each group should consist of **four/five students**.
- **Equal Contribution**: All members must contribute equally to:
  - Technical/coding tasks.
  - Experiments.
  - Report writing.
  - Presentation.
- The project should utilise **PyTorch**, **Python notebooks**, and **Google Colab**. Access to the **Surrey AI Supercomputer** is also available.

## Assessments: Marking Criterion

The group submission aspect of the project will be assessed as follows:

- **Technical Report (40% - 24 marks)**: Clarity, structure, critical analysis, results presentation, and discussion.
- **Functionality and Experimental Rigor (40% - 24 marks)**: Completeness of implementation across both tasks, multi-model comparison, ablation studies, localisation pipeline, statistical robustness, and performance of the models.
- **Code Quality (20% - 12 marks)**: Efficiency, modularity, reproducibility (e.g., fixed seeds, config files), and thorough documentation.

## Detailed Report Assessment (24 marks)

- Abstract: 2 marks
- Introduction (motivating both classification and localisation): 3 marks
- Literature Review (minimum 10 papers, covering ViT-based video models, attention mechanisms, action recognition benchmarks, and spatio-temporal action detection): 4 marks
- Methodology (architecture descriptions, training procedures, ablation design, localisation pipeline design): 6 marks
- Experiments and Results (multi-model comparison, ablation tables, localisation metrics, statistical analysis): 5 marks
- Interpretability and Error Analysis (visualisations, cross-task error correlation, qualitative discussion): 2 marks
- Conclusion and Future Work: 2 marks

## Functionality and Experimental Rigor (24 marks)

- Successful implementation of at least two classification architectures with correct adaptation: 5 marks
- Ablation studies (at least 3 factors, properly controlled): 4 marks
- Complete classification evaluation metrics (top-1, top-5, per-class F1, confusion matrix, statistical robustness): 4 marks
- Spatio-temporal localisation pipeline (architecture, training, tube generation): 5 marks
- Localisation evaluation (frame-mAP, video-mAP, qualitative visualisations): 3 marks
- Interpretability analysis (at least 2 methods from Part E, connecting to both tasks): 3 marks

## Code Quality (12 marks)

- Efficient, modular, and well-documented code adhering to best practices.
- Use of configuration files or argument parsers for reproducible experiments.
- Clear separation of data loading, model definition, training, evaluation, and visualisation code.
- Shared backbone code between the classification and localisation pipelines to avoid unnecessary duplication.